LeakGuard: Projeto e Implementação de um Sistema IoT para Detecção de Água, Monitoramento Remoto e Registro de Incidentes

Yaislin Bell Verdecia, José Antonio García Ocaña, and Martin Alonso Siffrin
Instituto Nacional de Telecomunicações (INATEL)
Santa Rita do Sapucaí, MG, Brazil
Emails: yaislin.verdecia@mtel.inatel.br, jose.ocana@mtel.inatel.br, martin.siffrin@mtel.inatel.br

***Resumo*—A detecção de água em locais sujeitos a infiltrações ou vazamentos requer resposta próxima ao ponto monitorado e mecanismos de acompanhamento remoto. Este trabalho apresenta o desenvolvimento do LeakGuard, um sistema de Internet das Coisas (IoT) que integra um ESP32, sensor resistivo, alarme acústico, comunicação Message Queuing Telemetry Transport (MQTT) e aplicativo Android nativo. O firmware realiza leitura digital com confirmação temporal de 400 ms para presença de água e 800 ms para retorno à condição seca, além de controlar o buzzer e manter um histórico circular de oito incidentes em memória não volátil. O protocolo MQTT intermedeia a publicação de estados e eventos e a recepção de comandos, enquanto o aplicativo disponibiliza monitoramento, controle remoto, estatísticas e registros persistentes em Room. A avaliação funcional caracterizou a resposta do sensor, o acionamento local e a troca de mensagens observada no MQTTX, complementados pela visualização dos dados no Android. A arquitetura resultante combina atuação independente da conexão de rede e supervisão distribuída, oferecendo uma solução extensível para o monitoramento pontual de presença de água.**

***Palavras-chave*—Internet das Coisas, ESP32, MQTT, detecção de água, Android, monitoramento remoto.**

***Abstract*—Water detection in environments exposed to leaks or seepage requires local response and remote monitoring capabilities. This paper presents the development of LeakGuard, an Internet of Things (IoT) system integrating an ESP32, a resistive water sensor, an audible alarm, Message Queuing Telemetry Transport (MQTT) communication, and a native Android application. The firmware implements digital sensing with a 400 ms confirmation interval for water presence and 800 ms for return to dry conditions, manages the buzzer, and maintains a circular history of eight incidents in non-volatile memory. MQTT supports state and event publication as well as remote commands, while the Android application provides monitoring, control, statistics, and persistent records using Room. Functional assessment covered sensor response, local alarm operation, and message exchange observed through MQTTX, together with data visualization on Android. The resulting architecture combines local operation independent of network connectivity with distributed supervision and provides an extensible basis for point-based water detection.**

***Index Terms*—Internet of Things, ESP32, MQTT, water detection, Android, remote monitoring.**

I. Introdução

A presença não identificada de água em ambientes residenciais, laboratórios e instalações técnicas pode comprometer equipamentos, instalações elétricas e materiais armazenados. A detecção localizada constitui uma medida de monitoramento preventivo, especialmente quando o ambiente permanece sem supervisão presencial. Entretanto, a eficácia de um aviso exclusivamente acústico diminui quando não há pessoas próximas ao ponto monitorado.

Soluções baseadas em Internet das Coisas (IoT) associam sensores físicos, processamento embarcado e serviços de comunicação, permitindo disponibilizar o estado de um ambiente em diferentes interfaces [1], [2]. Para sensores resistivos de contato, a informação obtida corresponde à presença de água na superfície instrumentada. A integração dessa leitura com alarmes, registros de incidentes e controle remoto amplia as possibilidades de acompanhamento sem alterar a natureza pontual da medição.

Nesse contexto, foi desenvolvido o *LeakGuard*, uma solução composta por três subsistemas: um nó ESP32 responsável pela detecção e sinalização locais; uma infraestrutura MQTT para troca bidirecional de mensagens; e um aplicativo Android para supervisão, comandos e consulta histórica. O processamento do evento permanece no microcontrolador, enquanto o aplicativo organiza informações correntes e registros recebidos em armazenamento próprio.

A contribuição do trabalho consiste na implementação integrada dessa arquitetura e na análise de seu comportamento funcional, contemplando a diferenciação entre estados seco e molhado, o gerenciamento do alarme, a comunicação entre clientes e a persistência de incidentes. A discussão também considera alternativas tecnológicas, potencial de integração e fatores necessários à evolução comercial do protótipo.

II. Fundamentação Teórica e Trabalhos Relacionados

A. Sistemas IoT aplicados à detecção de vazamentos

Sistemas de detecção de água utilizam diferentes grandezas físicas de acordo com a aplicação. Medições de pressão e vazão são particularmente relevantes para redes hidráulicas, enquanto sensores de contato atendem à identificação de água em superfícies delimitadas. Mohd Yussof e Ho [3] analisam métodos aplicados a edifícios inteligentes; Romero-Ben *et al.* [4] discutem estratégias de detecção e localização em redes de distribuição.

Entre as aplicações conectadas, Jan *et al.* [5] apresentam soluções para monitoramento de reservatórios, vazamentos e

acionamento de bombas. O LeakGuard adota uma abordagem orientada a eventos discretos: a leitura do sensor determina o estado de contato com água, que desencadeia sinalização local, publicação de informações e armazenamento do incidente. Dessa forma, a arquitetura concentra-se na resposta a ocorrências pontuais e na disponibilização de seu histórico.

B. Comunicação MQTT e processamento local

O protocolo Message Queuing Telemetry Transport (MQTT) adota o modelo de publicação e assinatura, no qual um *broker* encaminha mensagens aos clientes inscritos nos respectivos tópicos [6]. Esse desacoplamento permite integrar o ESP32, a aplicação Android e uma ferramenta de diagnóstico sem estabelecer conexões diretas entre todos os participantes.

No LeakGuard, a detecção e o acionamento do buzzer são executados localmente. O MQTT é responsável pela distribuição de telemetria, eventos de transição e comandos remotos. Mensagens retidas permitem que novos clientes recebam o estado mais recentemente publicado, enquanto o mecanismo Last Will and Testament (LWT) apoia a identificação de desconexões inesperadas. Esses recursos complementam os procedimentos de reconexão implementados no sistema [6].

C. Trabalhos relacionados e soluções comerciais

Detectores comerciais de água apresentam diferentes estratégias de conectividade e integração. O Aqara Water Leak Sensor T1 utiliza Zigbee 3.0 e depende de um *hub* compatível para funções conectadas [7]. O GoveeLife H5059, associado a um *gateway*, combina sinalização acústica e notificações por aplicativo [8]. Essas soluções exemplificam alternativas voltadas ao monitoramento residencial.

A Tabela I confronta características documentadas desses produtos com os recursos implementados no LeakGuard. A comunicação MQTT com tópicos definidos e o armazenamento histórico no Android constituem escolhas arquiteturais relevantes para a integração com outros clientes e para a consulta de incidentes, respectivamente. A comparação caracteriza funcionalidades e modelos de comunicação, sem constituir uma avaliação experimental entre dispositivos.

Tabela I
CARACTERÍSTICAS DOCUMENTADAS DE SOLUÇÕES DE DETECÇÃO DE ÁGUA.

Produto	**Conectividade**	**Supervisão e escopo**
Aqara T1 [7]	Zigbee 3.0 com *hub*.	Aplicativo e automação no ecossistema.
GoveeLife H5059 [8]	Sensor com *gateway* Wi-Fi.	Aviso acústico e alertas remotos.
LeakGuard	ESP32 / MQTT com broker.	Alarme, Android e histórico Room.

III. MATERIAIS E MÉTODOS

A. Arquitetura geral do sistema

A arquitetura foi organizada em três subsistemas interligados: o nó embarcado, o serviço de mensageria e a aplicação móvel. O ESP32 processa a leitura do sensor, controla o buzzer e utiliza armazenamento não volátil (NVS) para registrar incidentes locais. O *broker* EMQX encaminha a telemetria e os comandos MQTT. No Android, os estados recebidos alimentam a interface e os registros de eventos são armazenados em Room. O MQTTX Web atua como cliente de inspeção do fluxo de mensagens.

A Figura 1 explicita a organização dos componentes e a comunicação bidirecional entre o dispositivo e os clientes. O caminho sensor–ESP32–buzzer corresponde à cadeia de detecção e atuação; os canais MQTT transportam informações de estado, eventos e solicitações de controle.

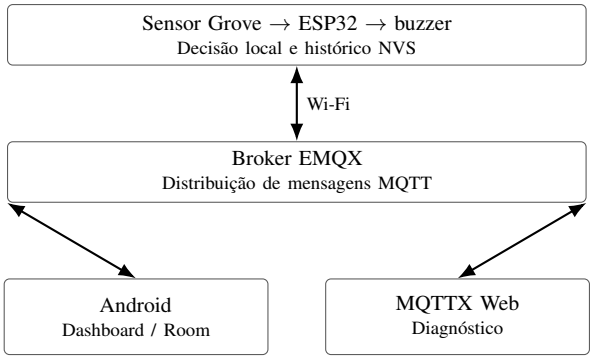


Fig. 1. Arquitetura do LeakGuard e separação entre operação local e clientes MQTT.

A separação entre atuação e comunicação mantém o tratamento de eventos no próprio nó embarcado. A conectividade acrescenta supervisão e gerenciamento remoto, mas não participa da decisão inicial de reconhecer a presença de água e acionar o aviso sonoro.

B. Caracterização dos componentes de hardware

A unidade de processamento é uma placa ESP32 DOIT DevKit V1, selecionada pela integração de recursos de computação embarcada e Wi-Fi em 2,4 GHz. O sinal digital do sensor foi associado ao GPIO34, enquanto o buzzer é controlado pelo GPIO23. No ESP32 clássico, os GPIO34–GPIO39 operam exclusivamente como entradas e não dispõem de resistores internos de polarização, razão pela qual o sinal de entrada deve apresentar nível lógico definido [9].

O Grove Water Sensor V1.1 utiliza pistas condutoras cujo contato com água altera a condição elétrica do circuito. A documentação do fabricante descreve polarização por *pull-up* fraco e alimentação nominal de 5 V [10]. O Grove Buzzer V1.2 fornece sinalização acústica comandada digitalmente, permitindo reproduzir sequências de tons por temporização no firmware [11]. A protoboard, os cabos e a alimentação USB completam a montagem experimental.

A Tabela II reúne a função de cada elemento e as restrições diretamente relacionadas ao seu emprego no protótipo. A seleção privilegia a integração de componentes disponíveis e a possibilidade de reconfiguração durante o desenvolvimento.

C. Montagem e integração eletrônica

O circuito emprega o GPIO34 como entrada do sensor de água e o GPIO23 como saída de controle do buzzer,

Tabela II
CARACTERIZAÇÃO DOS COMPONENTES FÍSICOS DO PROTÓTIPO.

Componente	Papel	Restrição principal
ESP32 DOIT DevKit V1	Controle, Wi-Fi e NVS.	GPIO34 somente entrada.
Grove Water Sensor V1.1	Contato resistivo.	Alimentação e polarização.
Grove Buzzer V1.2	Alerta acústico.	Potência sonora não medida.
Protoboard e cabos	Montagem e GND comum.	Contatos expostos.
USB e fonte	Energia e programação.	Sem bateria reserva.

com referência elétrica compartilhada. A Tabela III registra o mapeamento das conexões adotadas, incluindo alimentação e interface USB para programação do microcontrolador.

Tabela III
MAPEAMENTO DE SINAIS E ALIMENTAÇÃO DO PROTÓTIPO.

Origem	Destino	Função
Sensor (sinal)	GPIO34	Entrada digital.
Buzzer (controle)	GPIO23	Saída de atuação.
Módulos (GND)	GND comum	Referência elétrica.
Alimentação	3,3 V relatados	Tensão a conferir.
ESP32 (USB)	Fonte/computador	Energia e gravação.

A Figura 2 representa as interligações funcionais entre os módulos, identificando as linhas de sinal, alimentação e terra comum. O desenho sintetiza as conexões elétricas e complementa a fotografia do circuito apresentada nos resultados. No arranjo descrito, os módulos foram alimentados a 3,3 V, condição que exige atenção à especificação nominal do sensor e à compatibilidade de seus níveis lógicos com as entradas do ESP32.

D. Desenvolvimento do firmware embarcado

O firmware foi desenvolvido em C++ com o *framework* Arduino para ESP32 no ambiente PlatformIO. A biblioteca PubSubClient implementa o cliente MQTT e Preferences gerencia os dados persistidos em NVS [12], [13]. A aquisição digital utiliza nível lógico alto para indicar condição seca e baixo para contato com água, conforme observado na montagem.

A confirmação de presença de água requer permanência da condição detectada por 400 ms, enquanto o retorno à condição seca é confirmado após 800 ms. Essa histerese temporal reduz a influência de mudanças instantâneas do sinal sobre o registro de incidentes e introduz intervalos definidos de confirmação em cada transição.

A lógica mantém independentes o estado confirmado de presença de água, $w \in \{0, 1\}$, e o estado de silenciamento, $m \in \{0, 1\}$. O comando `SILENCE` estabelece $m = 1$ e `UNMUTE` restabelece $m = 0$. Desconsiderando a cadência dos tons, a habilitação do aviso sonoro pode ser expressa por

$$a = w(1 - m). \tag{1}$$

Assim, a Eq. (1) representa o controle do buzzer sem alterar a condição de presença de água. A geração sonora baseada em `millis()` permite manter a aquisição e o tratamento de mensagens durante a execução da sequência acústica.

Ao término de um incidente, são atualizados a contagem de ocorrências, a duração acumulada e o histórico circular dos oito registros mais recentes. As marcas temporais são associadas às informações obtidas por Network Time Protocol (NTP); registros sem tempo absoluto válido são identificados pelo valor Unix zero. A Figura 3 apresenta a sequência de amostragem, confirmação temporal, atualização de estados e execução das funções de sinalização e registro.

E. Comunicação MQTT e gerenciamento de eventos

A comunicação utiliza o *broker* público `broker.emqx.io`, com MQTT 3.1.1. O ESP32 e o aplicativo Android estabelecem conexões TCP pela porta 1883, enquanto o MQTTX Web opera por WebSocket seguro na configuração descrita. Os tópicos compartilham o prefixo `leakguard/jgocana-yverdecia-iot-device01`, utilizado para identificar o dispositivo no ambiente de demonstração [14], [15].

A Tabela IV apresenta a organização dos cinco tópicos e seus respectivos papéis. As mensagens de `/state` descrevem condições e estatísticas do dispositivo; `/event` sinaliza o início e o encerramento de incidentes; e `/cmd` recebe solicitações dos clientes. O canal `/history` disponibiliza os registros locais e `/connection` informa o estado de conectividade.

Tabela IV
CONTRATO DE PUBLICAÇÃO E ASSINATURA MQTT DO LEAKGUARD.

Sufixo	Direção	Conteúdo
`/state`	ESP32 → cliente	Sensor, buzzer, contagens e durações.
`/event`	ESP32 → cliente	`LEAK_START` e `LEAK_END`.
`/history`	ESP32 → cliente	Oito incidentes recentes.
`/connection`	ESP32 / broker	`online` / `offline`.
`/cmd`	cliente → ESP32	Controle e consultas.

A telemetria é publicada aproximadamente a cada dois segundos no formato JavaScript Object Notation (JSON), contendo os campos `water_detected`, `buzzer`, `muted`, `leak_count`, `current_leak_seconds`, `total_leak_seconds` e `updated_at_unix`. Os comandos `SILENCE` e `UNMUTE` controlam o aviso acústico, enquanto `GET_STATUS` e `GET_HISTORY` solicitam atualização do estado e do histórico. Mensagens retidas são utilizadas nos tópicos de estado, histórico e conexão; o mecanismo LWT disponibiliza a indicação `offline` em caso de desconexão inesperada do ESP32, após sua detecção pelo *broker* [6].

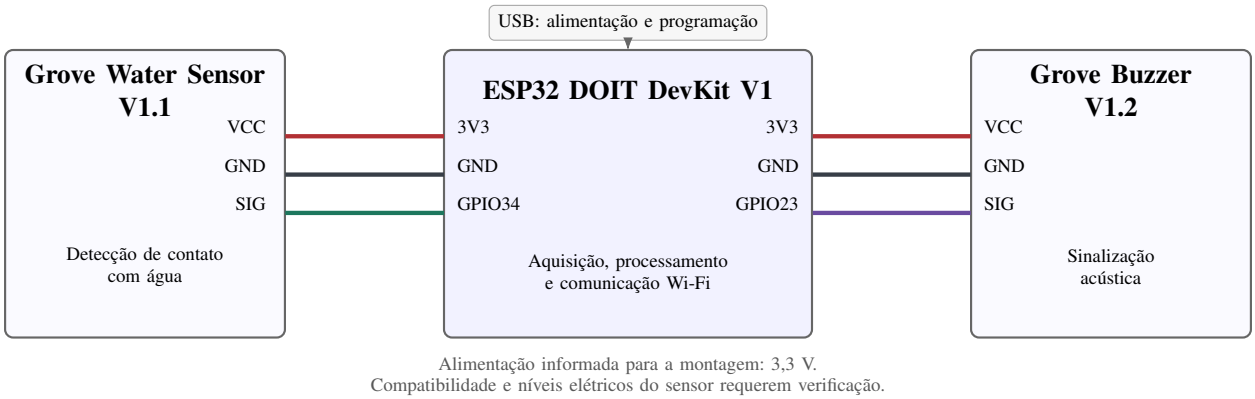


Fig. 2. Diagrama de interligação funcional do LeakGuard: sensor em GPIO34, buzzer em GPIO23, 3,3 V e GND comum. A disposição dos terminais é esquemática e não indica a posição física dos pinos.

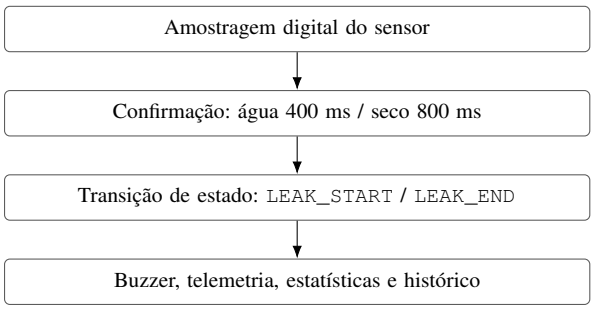


Fig. 3. Fluxo lógico da detecção e do processamento de incidentes no ESP32.

F. Desenvolvimento da aplicação Android

O aplicativo Android foi desenvolvido em Kotlin com Jetpack Compose e Material Design 3. Sua organização emprega separação de responsabilidades inspirada em Clean Architecture e no padrão Model–View–ViewModel (MVVM). O cliente MQTT HiveMQ implementa as operações de conexão, assinatura e publicação, enquanto Coroutines e StateFlow distribuem as atualizações recebidas aos componentes de interface.

O painel de supervisão apresenta o estado da detecção, o funcionamento e o silenciamento do buzzer, as estatísticas e a conectividade do dispositivo. A camada de apresentação distingue a conexão do telefone ao *broker*, a disponibilidade do ESP32 e a atualidade da última telemetria. Um intervalo de aproximadamente dez segundos sem nova mensagem válida caracteriza informação desatualizada, preservando a última leitura para consulta com identificação apropriada.

Para o armazenamento histórico, a aplicação utiliza Room, biblioteca de persistência relacional local [16]. O dispositivo conserva os oito incidentes mais recentes, enquanto o Android agrega os registros recebidos e permite sua consulta posterior. A identificação composta pelo dispositivo e pelo incidente é utilizada na deduplicação durante a recuperação de mensagens retidas, eventos de encerramento e respostas a `GET_HISTORY`. DataStore armazena parâmetros de conexão e preferências da interface [17]. As estatísticas móveis são calculadas a partir dos registros persistidos no telefone, distinguindo-se dos contadores mantidos no microcontrolador.

A Figura 4 apresenta a organização lógica da aplicação. Os componentes Compose consomem estados expostos pelo ViewModel, ao passo que os repositórios concentram acesso MQTT, persistência Room e configurações DataStore. Essa estrutura estabelece a separação entre apresentação, regras de processamento e fontes de dados.

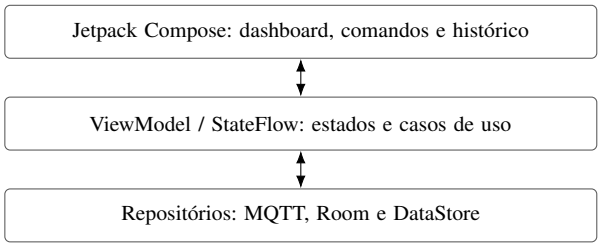


Fig. 4. Organização lógica da aplicação Android e de suas fontes de dados.

G. Procedimento de avaliação experimental

A avaliação foi estruturada em cenários de interação entre entradas físicas, comandos de controle e respostas observáveis. O procedimento considera a alternância entre condições seca e molhada, a emissão de comandos MQTT e a inspeção do estado apresentado no dispositivo e na aplicação. As saídas analisadas incluem a atuação do buzzer, a telemetria, as mensagens de evento e os registros históricos.

O ciclo funcional compreende a leitura inicial em condição seca, a aplicação de água ao sensor, a confirmação do incidente, o silenciamento e a reativação do buzzer, a secagem e a atualização dos registros. A recuperação do histórico e o tratamento de reconexões compõem a avaliação da integração entre os subsistemas. A Tabela V organiza os cenários e os critérios utilizados para observar cada resposta funcional.

A avaliação é qualitativa e tem como unidade de observação as transições de estado e as mensagens produzidas pelo sistema. Os intervalos de 400 e 800 ms representam configurações da lógica de confirmação temporal, enquanto o exame de latência fim a fim, consumo energético ou confiabilidade estatística

Tabela V
CENÁRIOS E RESPOSTAS FUNCIONAIS CONSIDERADAS NA AVALIAÇÃO.

Ensaio	**Resposta funcional observável**
Seco ↔ molhado	Transição confirmada, alarme e eventos.
`SILENCE`/`UNMUTE`	Mudança de aviso, sem apagar a fuga.
`GET_STATUS`	Nova publicação em `/state`.
`GET_HISTORY`	Histórico em `/history`.
Reinício do ESP32	Recuperação de registros persistidos.
Desconexão	Alarme local; sinalização de indisponibilidade.
Reconexão Android	Sincronização e deduplicação em Room.

requer instrumentação e repetição controlada. Esses aspectos são considerados na análise das limitações do protótipo.

IV. RESULTADOS E VALIDAÇÃO DO SISTEMA

A. Funcionamento do dispositivo embarcado

A leitura digital diferenciou as condições de superfície seca e contato com água por níveis lógicos alto e baixo, respectivamente. Após a confirmação da presença de água, o firmware habilita o sinal acústico e registra a transição correspondente. O comando `SILENCE` permite interromper a emissão sonora mantendo o incidente ativo, e o retorno ao estado seco encerra o aviso e prepara o sistema para uma nova ocorrência.

A montagem utilizada está documentada na Figura 5. A fotografia identifica a placa ESP32, o sensor Grove, o buzzer e suas interconexões na protoboard, permitindo relacionar a implementação física ao mapeamento funcional da Figura 2.

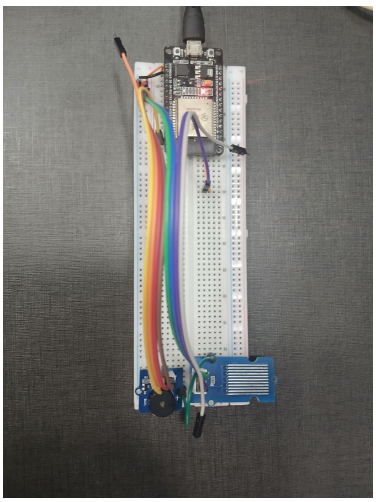


Fig. 5. Montagem real do nó LeakGuard em protoboard: ESP32, Grove Water Sensor V1.1 e Grove Buzzer V1.2.

B. Comunicação e controle remoto via MQTT

A troca de mensagens entre ESP32 e os clientes MQTT foi acompanhada por meio do MQTTX Web. As publicações em `/state` disponibilizam a condição do sensor, o estado do buzzer e os campos de estatísticas; o envio de `SILENCE` por `/cmd` demonstra o canal de controle remoto estabelecido pelo protocolo.

A Figura 6 reúne recortes da inspeção realizada no MQTTX, mostrando telemetria de estado e a publicação e recepção do comando de silenciamento. A identificação dos tópicos e dos conteúdos permite examinar o intercâmbio de mensagens independentemente da aplicação Android.

C. Funcionamento da aplicação Android

O aplicativo Android apresenta os dados em interfaces destinadas à supervisão imediata e ao acompanhamento dos incidentes. No *dashboard*, são exibidos a condição do sensor, os controles do buzzer e os indicadores de conectividade. A visualização complementar organiza contagens, durações e eventos recentes, fornecendo uma perspectiva histórica das informações registradas.

A Figura 7 apresenta essas interfaces: a tela (a) concentra o monitoramento e as ações de controle, enquanto a tela (b) reúne estatísticas e ocorrências recentes. As capturas mostram como a informação proveniente do nó embarcado é organizada para consulta pelo usuário, preservando a distinção entre o estado instantâneo e os indicadores acumulados.

D. Avaliação integrada e comportamento diante de interrupções

A integração estabelece dois fluxos complementares: o ESP32 realiza a aquisição, a confirmação dos incidentes e a sinalização acústica; a infraestrutura MQTT e o Android acrescentam distribuição de mensagens, comandos e acesso ao histórico. Essa separação permite que o processamento local prossiga durante interrupções da comunicação, desde que o nó permaneça alimentado.

Na camada móvel, os estados de conexão e a atualização da telemetria são tratados de modo independente. Após uma reconexão, o aplicativo pode recuperar os incidentes ainda disponíveis no histórico do ESP32 e incorporar novos registros ao banco Room por meio da deduplicação. O histórico já sincronizado permanece acessível localmente, mesmo quando não há conectividade com o *broker*.

V. DISCUSSÃO, BENEFÍCIOS E VIABILIDADE COMERCIAL

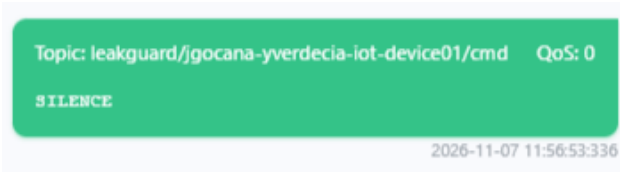
A. Análise dos resultados e contribuições

A arquitetura implementada evidencia a complementaridade entre processamento embarcado e supervisão remota. A confirmação e o acionamento do alarme ocorrem junto à fonte da medição, reduzindo a dependência de serviços externos para a sinalização local. A mensageria MQTT amplia o acesso às informações e permite compartilhar estados e comandos entre diferentes clientes.

A combinação de NVS no ESP32 e Room no Android atende a escalas distintas de retenção. O armazenamento circular no nó mantém incidentes recentes próximos ao sensor, enquanto o aplicativo acumula registros sincronizados e oferece consulta histórica. Essa organização favorece a continuidade das informações já recebidas e a recuperação de ocorrências após interrupções de comunicação, dentro da capacidade de armazenamento disponível no dispositivo.



(a) Telemetria publicada em `/state`



(b) Publicação de `SILENCE`

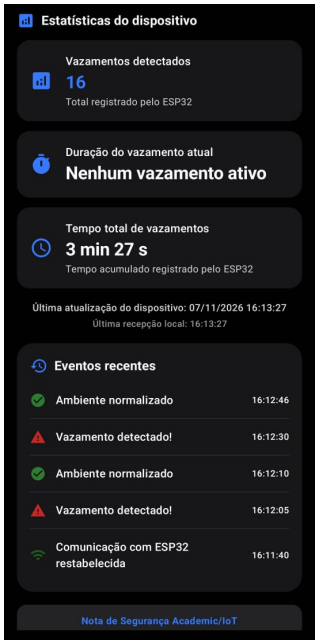


(c) Mensagem no tópico `/cmd`

Fig. 6. Recortes de uma mesma captura original do MQTTX Web, evidenciando a telemetria de estado (a) e o intercâmbio do comando `SILENCE` (b)–(c).



(a) Monitoramento e comandos



(b) Estatísticas e eventos recentes

Fig. 7. Capturas reais da interface Android do LeakGuard: estado instantâneo do dispositivo (a) e indicadores acumulados com eventos recentes (b).

B. Benefícios da solução IoT

O LeakGuard associa ao aviso acústico tradicional funções de supervisão e controle remoto. A visualização do estado do sensor e do buzzer, a consulta a eventos e os indicadores históricos apoiam o acompanhamento de ocorrências sem necessidade de inspeção presencial contínua. A disponibilidade de tópicos MQTT documentados também simplifica a integração com clientes externos, enquanto o MQTTX permite inspecionar o tráfego independentemente da interface Android.

A utilização de componentes embarcados de ampla disponibilidade e de um aplicativo móvel nativo favorece a reprodução do sistema em contextos de ensino e prototipagem. Seu benefício imediato está na integração entre detecção localizada, atuação acústica e rastreabilidade dos incidentes.

C. Análise de viabilidade comercial

A evolução do LeakGuard para um produto comercial envolve custos de aquisição de componentes, montagem, alimentação, proteção física, infraestrutura de comunicação e manutenção. A Tabela VI sintetiza as principais categorias que compõem essa análise e suas implicações técnicas e operacionais. A avaliação econômica apresentada é qualitativa, orientada à identificação dos fatores que condicionam a produção e a operação do dispositivo.

A arquitetura demonstra viabilidade de implementação com módulos disponíveis comercialmente, mas sua conversão em produto requer projeto de placa e invólucro, qualificação elétrica, estimativa do custo unitário e definição de serviços de suporte. As alternativas Aqara e GoveeLife discutidas na Seção

Tabela VI
CATEGORIAS DE CUSTOS E REQUISITOS PARA EVOLUÇÃO COMERCIAL DO SISTEMA.

Item	Aspecto econômico e operacional
ESP32, sensor e buzzer	Custo unitário dos módulos eletrônicos.
Placa, cabos e fonte	Materiais de interconexão e consumo elétrico.
Caixa e proteção	Invólucro, proteção e conformidade.
Montagem e ensaios	Processo produtivo e qualificação.
Infraestrutura de rede	Serviços, segurança e disponibilidade.
Distribuição e pós-venda	Custos recorrentes e atendimento.

II indicam a existência de soluções conectadas no mercado e oferecem parâmetros de funcionalidade para posicionamento do LeakGuard.

D. Integração com outras soluções e limitações

A utilização de MQTT oferece uma interface de comunicação que pode ser estendida a plataformas de automação residencial, painéis externos e sistemas com vários nós de detecção. Uma implantação em múltiplas zonas demandaria identificação estável de dispositivos, organização de tópicos por local monitorado e tratamento centralizado ou distribuído das ocorrências.

As restrições atuais concentram-se em quatro aspectos. No hardware, o sensor resistivo está sujeito a desgaste e a montagem em protoboard não oferece proteção para ambientes úmidos, além de não haver alimentação de reserva. Na comunicação, a utilização de um *broker* público pela porta TCP 1883 expõe a necessidade de autenticação, controle de acesso e cifragem Transport Layer Security (TLS). Na persistência, a capacidade circular de oito eventos limita a recuperação de incidentes não sincronizados. Por fim, a disponibilidade de notificações no Android depende das permissões e das condições de execução do aplicativo. O enfrentamento dessas restrições orienta as etapas de robustecimento do sistema.

VI. CONCLUSÕES E TRABALHOS FUTUROS

O desenvolvimento do LeakGuard resultou na integração de um nó ESP32 com detecção resistiva de água e alarme acústico, comunicação MQTT bidirecional e aplicação Android destinada ao monitoramento e ao registro de incidentes. A avaliação funcional apresentou a distinção entre condições seca e molhada, o gerenciamento do aviso local, o intercâmbio de mensagens e a organização das informações nas interfaces móveis.

A separação entre o processamento local e os serviços de comunicação constitui o principal resultado de arquitetura. O armazenamento em duas camadas combina a retenção dos eventos recentes no dispositivo com a consulta histórica dos registros recebidos pelo Android, ampliando as possibilidades de acompanhamento das ocorrências.

Como continuidade, destacam-se a avaliação quantitativa de tempos de resposta e repetibilidade, a adoção de alimentação de reserva e encapsulamento adequado, o reforço da segurança MQTT e a expansão para múltiplos pontos de detecção. Essas etapas permitirão investigar o comportamento do sistema em períodos prolongados de operação e em contextos de instalação mais exigentes.

REFERÊNCIAS

[1] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[3] N. A. Mohd Yussof and H. W. Ho, "Review of water leak detection methods in smart building applications," *Buildings*, vol. 12, no. 10, p. 1535, 2022.

[4] L. Romero-Ben, D. Alves, J. Blesa, G. Cembrano, V. Puig, and E. Duviella, "Leak detection and localization in water distribution networks: Review and perspective," *Annual Reviews in Control*, vol. 55, pp. 392–419, 2023.

[5] F. Jan, N. Min-Allah, S. Saeed, S. Z. Iqbal, and R. Ahmed, "IoT-based solutions to monitor water level, leakage, and motor control for smart water tanks," *Water*, vol. 14, no. 3, p. 309, 2022.

[6] OASIS, "MQTT Version 3.1.1," OASIS Standard, Oct. 29, 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html, 2014.

[7] Aqara, "Water leak sensor T1: Specifications and frequently asked questions," [Online]. Available: https://www.aqara.com/en/water-leak-sensor-t1/water-leak-sensor-t1-specs and https://www.aqara.com/en/water-leak-sensor-t1/water-leak-sensor-t1-faq, accessed: Nov. 7, 2026.

[8] Govee, "GoveeLife smart water leak detector with Wi-Fi gateway," [Online]. Available: https://us.govee.com/products/smart-water-leak-detector, accessed: Nov. 7, 2026.

[9] Espressif Systems, "ESP32 Series Datasheet," [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.html, accessed: Nov. 7, 2026.

[10] Seeed Studio, "Grove – Water Sensor: Product documentation," [Online]. Available: https://wiki.seeedstudio.com/Grove-Water_Sensor/, accessed: Nov. 7, 2026.

[11] ——, "Grove – Buzzer: Product documentation," [Online]. Available: https://wiki.seeedstudio.com/Grove-Buzzer/, accessed: Nov. 7, 2026.

[12] N. O'Leary, "PubSubClient: An Arduino client for MQTT," [Online]. Available: https://github.com/knolleary/pubsubclient, accessed: Nov. 7, 2026.

[13] Espressif Systems, "Preferences — Arduino-ESP32 documentation," [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/preferences.html, accessed: Nov. 7, 2026.

[14] EMQ Technologies, "Free public MQTT broker," [Online]. Available: https://www.emqx.com/en/mqtt/public-mqtt5-broker, accessed: Nov. 7, 2026.

[15] ——, "MQTTX Web: Get started," [Online]. Available: https://mqttx.app/docs/web/get-started, accessed: Nov. 7, 2026.

[16] Android Developers, "Save data in a local database using Room," [Online]. Available: https://developer.android.com/training/data-storage/room, accessed: Nov. 7, 2026.

[17] ——, "Datastore," [Online]. Available: https://developer.android.com/topic/libraries/architecture/datastore, accessed: Nov. 7, 2026.